Real-World Engineering Challenges #7: Choosing Technologies

Choosing frameworks, languages and architecture approaches at Trello, Birdie, MetalBear and Motive.

GERGELY OROSZ
NOV 29, 2022

👋 *Hi, this is Gergely with the monthly free issue of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at big tech and high-growth startups through the lens of engineering managers and senior engineers. Subscribe to get weekly issues. Many subscribers expense this newsletter to their learning and development budget.* 👇

'Real-world engineering challenges' *is a series in which I interpret interesting software engineering or engineering management case studies from tech companies. You might learn something new in these articles, as we dive into the concepts they contain.*

I've taken a slightly different approach from previous articles in this series by reaching out with questions to the authors of interesting and relevant engineering blog posts, in order to share previously unpublished details and learnings from them.

Today, we cover:

1. **Trello choosing Kafka over RabbitMQ for messaging.** Trello used RabbitMQ to power its websockets functionality for several years. However, after noticing reliability issues and high resource usage, the team decided to solve this – and evaluated five alternatives.
2. **Why Birdie moved to Micro Frontends.** Birdie is a complex web app for healthcare providers. The team was frustrated by tests running slowly during each

change, so they investigated how to modularize their codebase and reduce the tight coupling between parts of it.

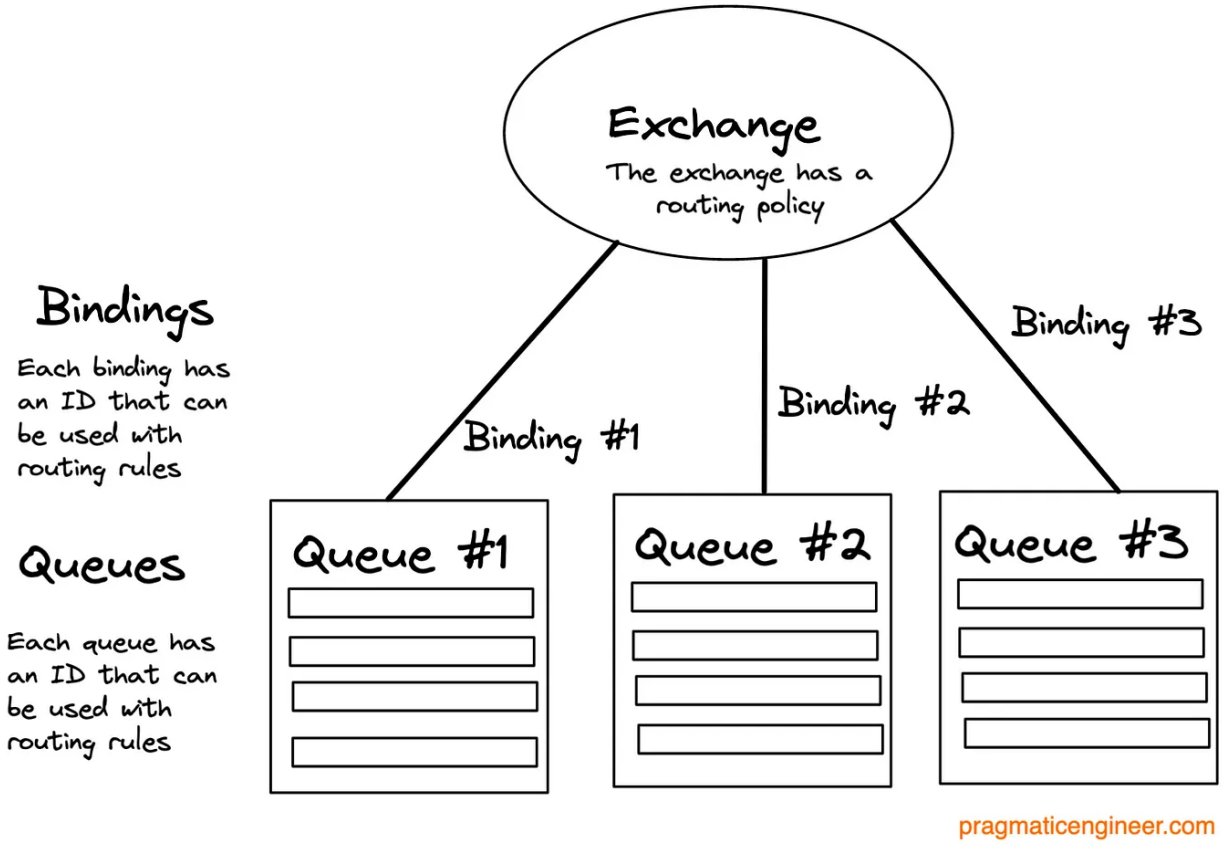
3. **Why MetalBear settled on Rust.** As a six month-old company, MetalBear had the option of choosing pretty much any programming language it wanted for its stack, and selected Rust. On top of performance, hiring considerations were part of the decision.
4. **Why Motive moved over to Kotlin Multiplatform Mobile (KMM.)** The team built the Motive Fleet application for transportation businesses, for iOS and Android. However, iOS was perpetually 1-2 months behind, and the business logic was slightly different from Android. The team decided to explore alternative approaches for sharing business logic between iOS and Android.

1. Trello Choosing Kafka over RabbitMQ

To power its websockets functionality, Trello used a Redis Pub/Sub implementation until 2015, then RabbitMQ from 2015-2018. In 2018, there was a problem of network partitioning with RabbitMQ which forced a reevaluation of the technology choice.

A short overview of RabbitMQ. To understand the problem the Trello team faced with RabbitMQ, we need to first understand some basics about RabbitMQ:

- **Exchange**: this is the entry point to which messages are published.
- **Queue:** exchanges are linked to one or more message queues. A message queue is exactly as it says.
- **Binding**: the connection between the exchange and the queues. Bindings can have binding keys.
- **Routing policy**: the strategy on how the exchange handles routing.



How exchanges, bindings, queues and routing policies are connected in RabbitMQ

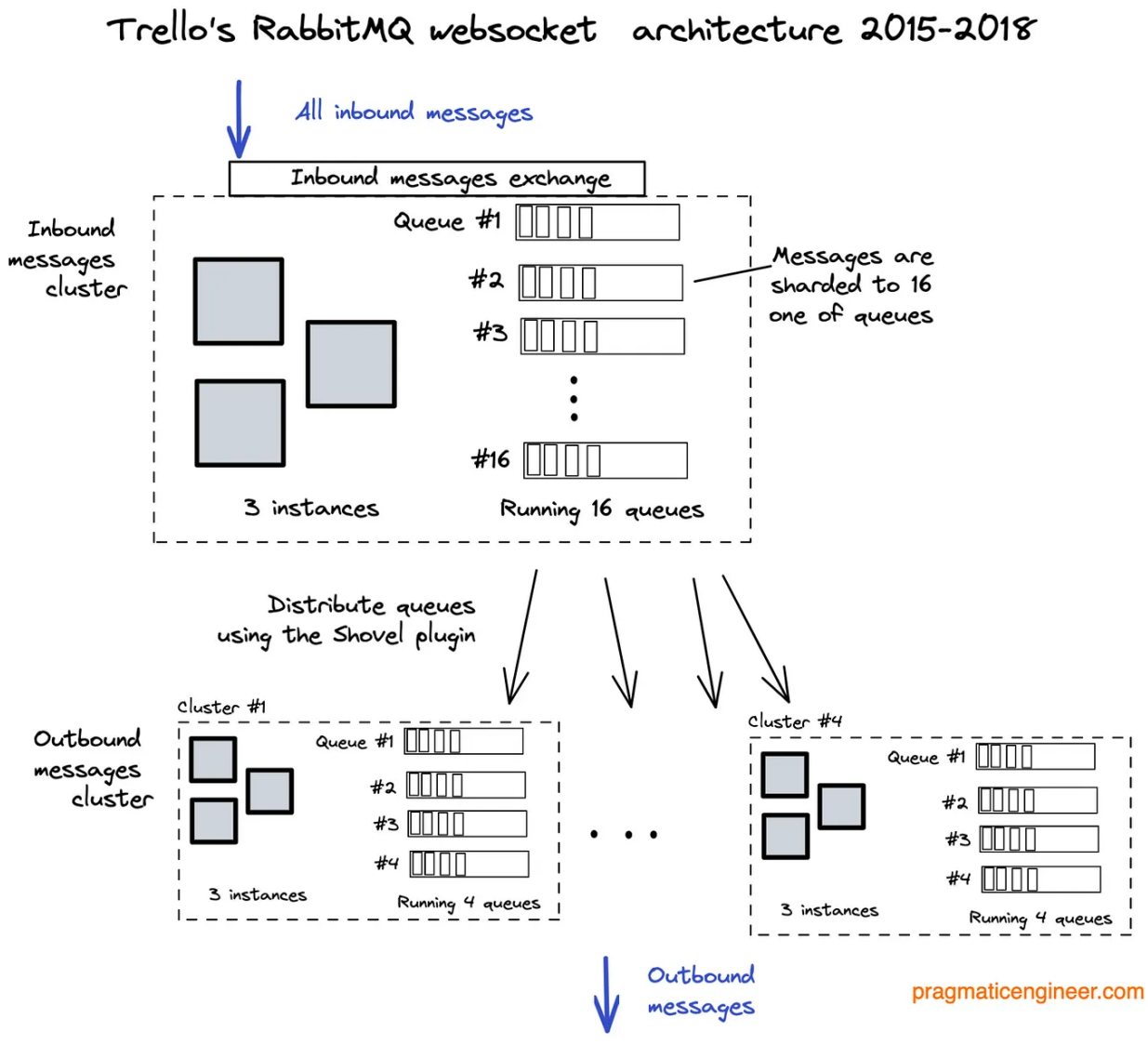
There are four common routing policies which RabbitMQ supports:

- *Fanout*: sends all inbound messages to the exchange and all queues bound to it, regardless of routing keys. This approach ignores routing keys.
- *Direct*: the message is sent to queues for which the routing key matches the binding key.
- *Topic*: allows wildcat matching between the routing key and the binding key. For example, you could set a topic of “tasks” and this would route to both the “tasks.important” and “tasks.unimportant” bindings.
- *Headers*: uses the header of a request to decide routings.

Queues can be transient in RabbitMQ, which means they can be destroyed as soon as the TCP connection that created them closes.

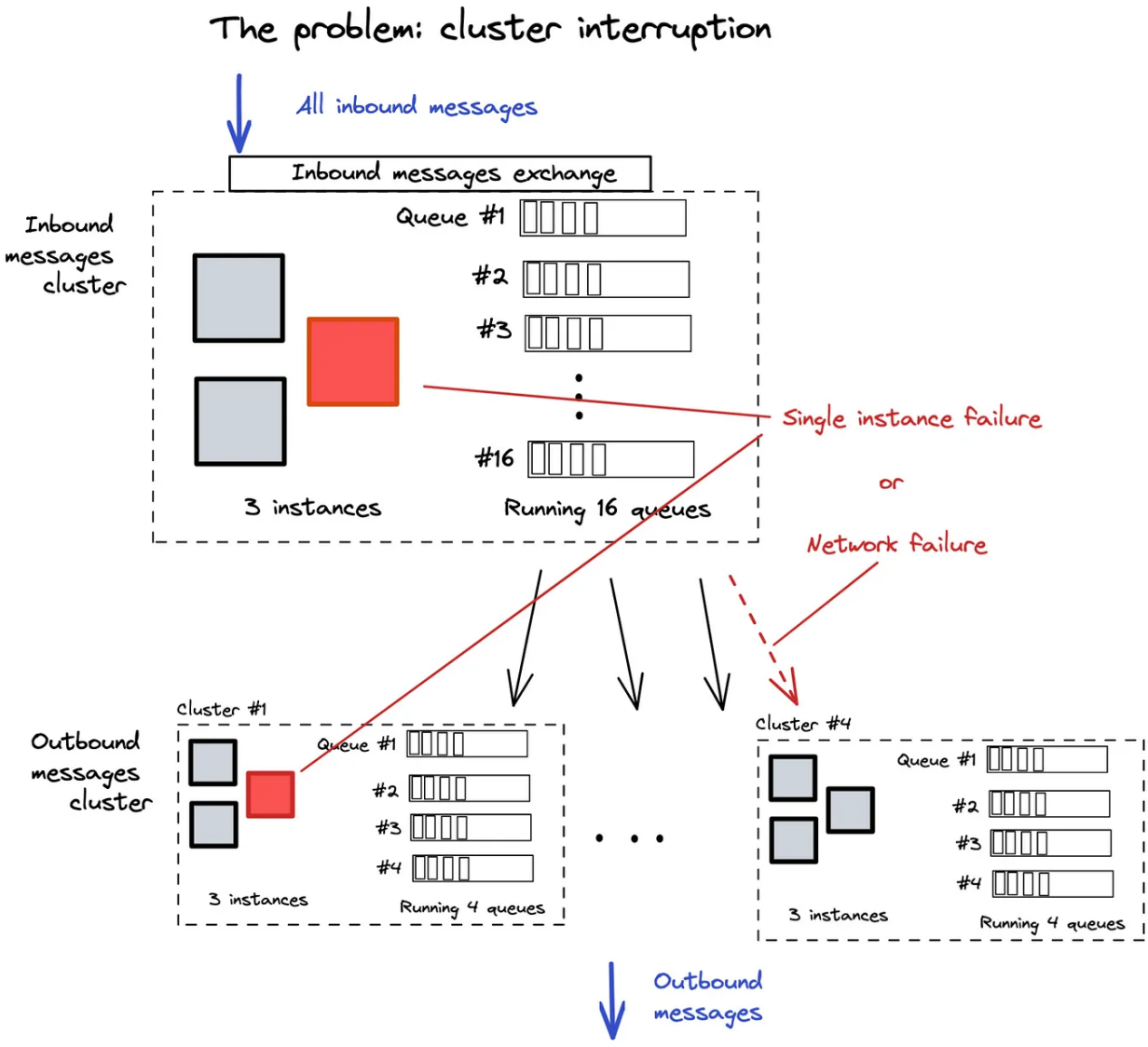
Trello's websocket implementation supported subscribing and unsubscribing to and from a notifications channel, which can be for a Trello board, a member, an organization, a card, or other Trello data models. Model_id was used as an identifier.

Original architecture. Trello used RabbitMQ to shard inbound messages to one of 16 queues. Behind the scenes, they ran 3 instances to handle inbound messages distributing to one of 16 queues. Then, they used RabbitMQ's Shovel plugin to map these 16 queues to 4 outbound clusters. Each outbound message cluster ran on 3 instances, and handled 4 outbound queues.



Trello's websocket architecture, built on RabbitMQ, as it worked 2015-2018.

The problem: cluster interruptions and performance. The Trello team observed major problems when a cluster went down due to a network interruption, or when a single member of a cluster went down. When this happened, they had to do a full reset; drop all sockets, and force the web clients to reconnect. Even worse, messages were lost during this full reset.



The problems with this infrastructure were how data was lost on single instance failures or network failures.

The other problem was how slow and resource intensive it was to create a queue and a binding in RabbitMQ. During a full reset, it took considerable time for these queues and bindings to be created.

Alternatives. So, the Trello team searched for alternatives to RabbitMQ and evaluated each solution based on their requirements, such as:

- Failover capabilities
- In-order message delivery per shard
- Supports fanout message distribution
- Low latency
- Supports required throughput of 2,000 messages/second

The team explored these 5 messaging alternatives:

1. Kafka
2. Amazon SNS (Simple Notification Service) + SQS (Simple Queue Service)
3. Amazon SNS + FIFO (first-in-first-out) SQS
4. Amazon Kinesis: a serverless streaming data service
5. Redis Streams

After evaluating the options, the team found Kafka and Redis Streams fitted their requirements and chose Kafka because Redis Streams was still in an unstable state: the Streams functionality was only committed in a branch that was marked as 'unstable'. The Trello team rebuilt the websocket architecture on top of Kafka, and now use a master-client architecture. Read the full article for more detail on the current setup, how unexpectedly cheaper Kafka is to operate, and an outage they experienced after switching over:

It's interesting to be reminded that a technology's pain points may not reveal themselves for months, or even years. The Trello team moved over to RabbitMQ after years of using a Redis Pub/Sub solution. So, why did they move away from it? I asked software engineer Sebastian Mayr – who wrote the article – and he shared that the biggest problem they had was that clustered Redis did not guarantee delivery in the event of a network issue or a failover.

As Trello grew, the problem of nodes failing became more apparent, as did the hardware costs of operating the Websocket infrastructure. After three years, the Trello team decided to explore if they could solve both problems with a different messaging service.

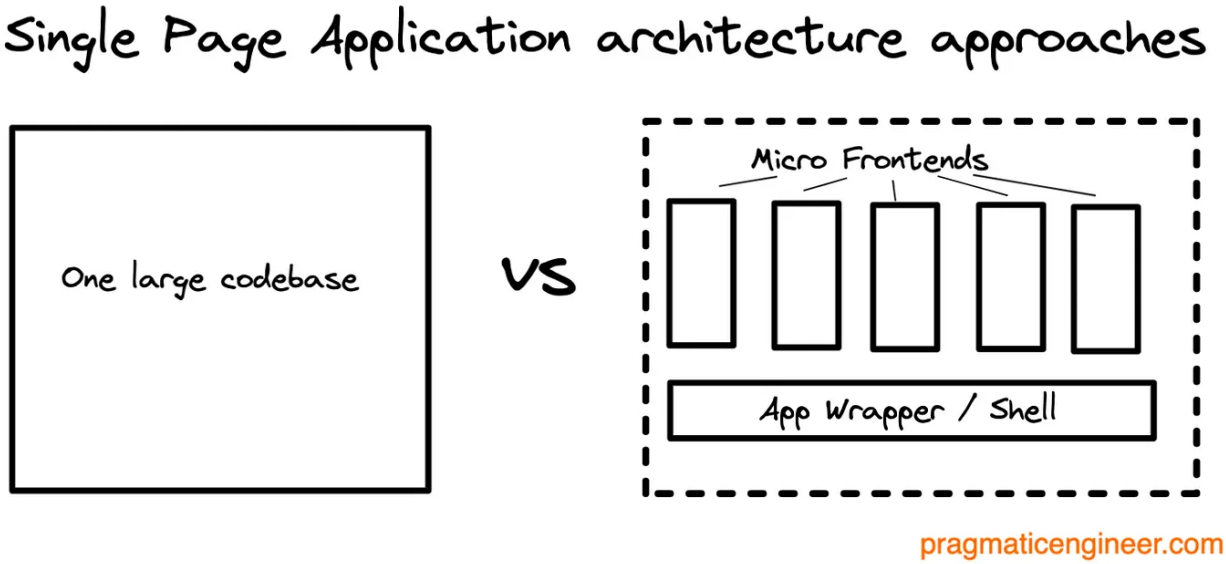
I liked how thoroughly the engineering team evaluated a variety of messaging options. They listed a variety of alternatives and looked hard at each one, based on their own requirements and pain points.

What I was missing from Sebastian's article was detail about the migration itself; I can only assume this work was non-trivial. One thing which makes such a migration easier is that at least it's not a data migration. So, I'd assume the team could test the new architecture in action by shadowing the functionality, or by rolling out to a growing number of users. We cover migration approaches in both the Real-world engineering challenges #6 and Migrations done well articles.

2. Why Birdie moved to Micro Frontends

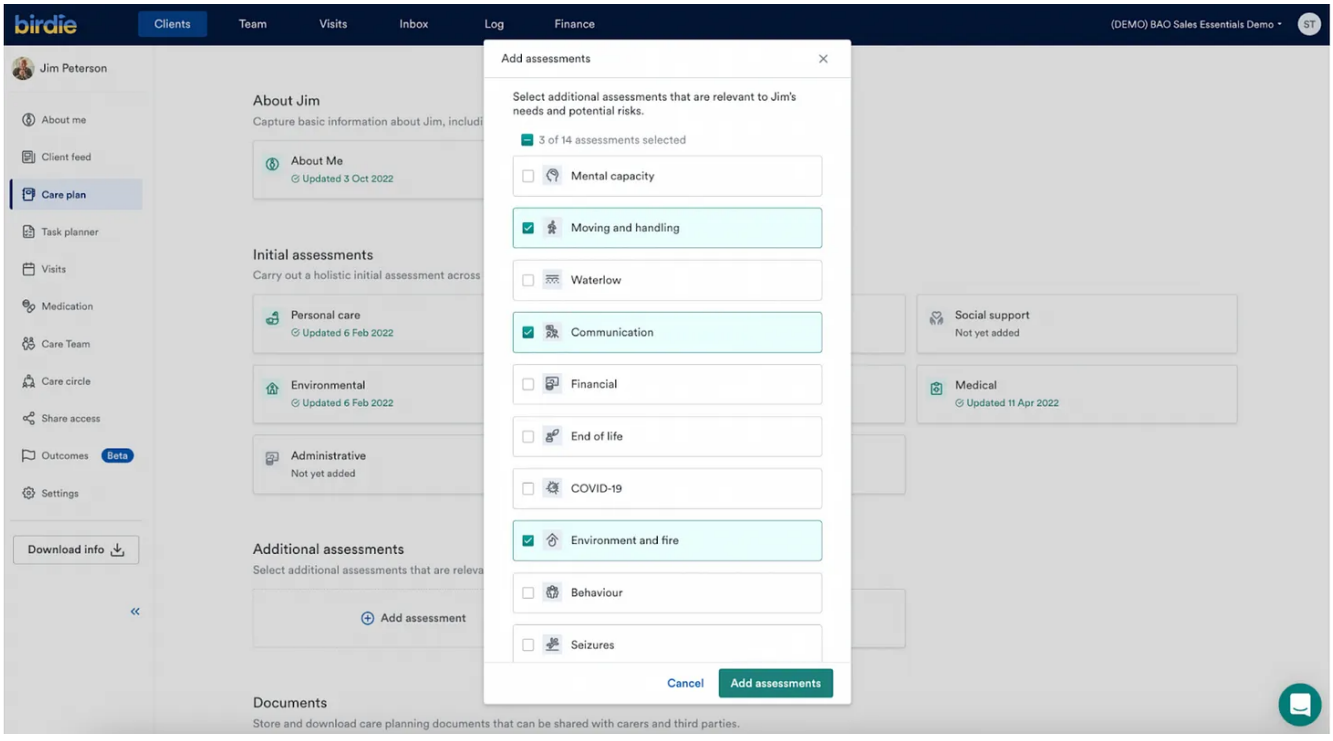
Birdie is a home healthcare technology platform, headquartered in London. The company raised a $30M Series B in June this year. They shared the journey of moving to Micro Frontends in this recent engineering blog post. I talked with the author of this post, Steve Heyes, who's a software engineer at Birdie, for more detail.

Micro Frontends are an architectural pattern somewhat similar to microservices. Instead of keeping the codebase of a Single Page Application (SPA) all in a single monolithic codebase, Micro Frontends split the codebase into separate components which are held together by a shell:



Micro Frontends are an architectural approach for structuring web applications.

The Birdie team has a Single Page Application built in React, talking to an API on the backend. Here's a screenshot of what the application looks like:



The Birdie React app. It's an app built for home healthcare use cases.

As the app grew, the engineering team noticed a problem growing more visible: tests took a long time to run. For each and every change, all tests in the codebase had to run, including a small number of unit tests, lots of integration tests, and a bunch of end-to-end tests using Cypress. This high quantity of automated tests started to slow down development, which is when the team came up with the idea of dividing the app into independent pieces using Micro Frontends, which could also be tested independently.

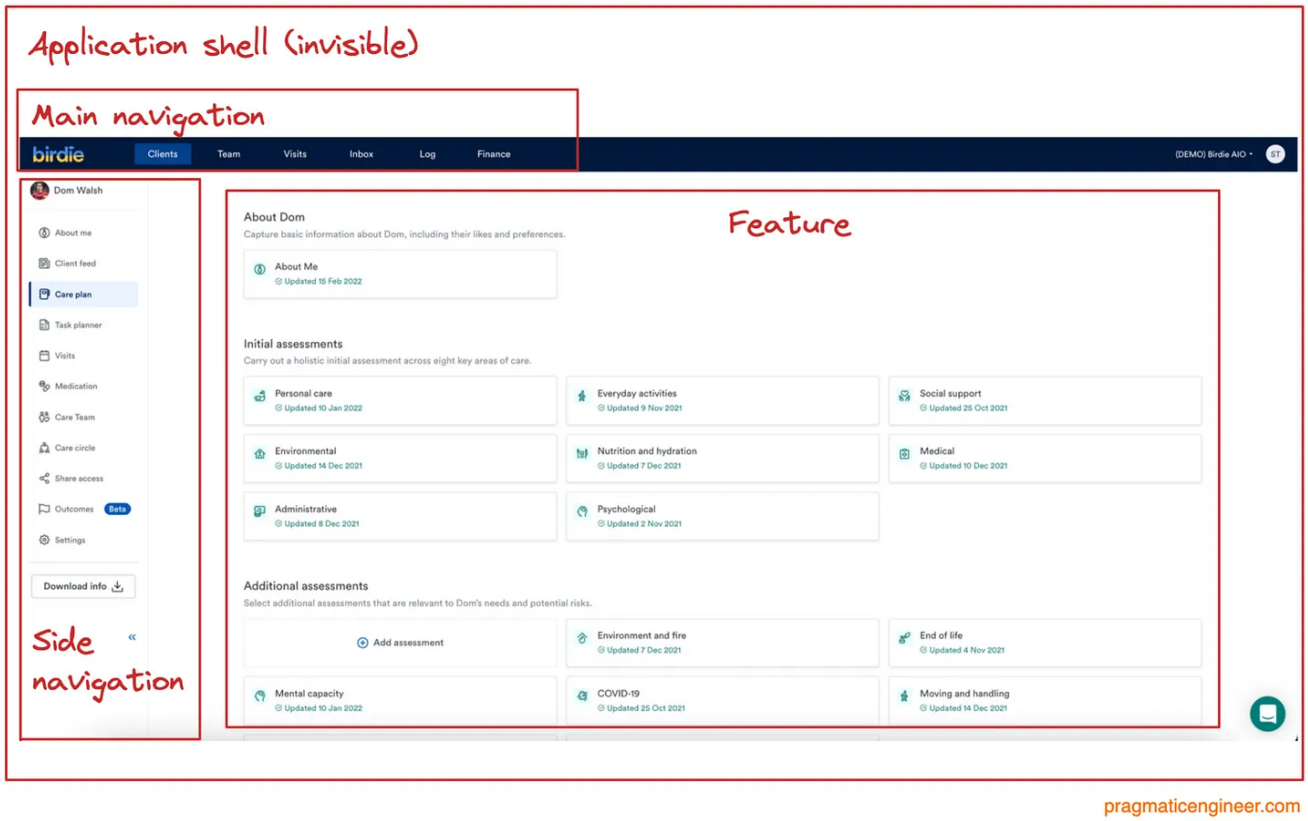
Steve was kind enough to share more details about this adoption process. He says:

'In July 2021, the team ran an "engineering improvement sprint." The goal was to fix platform debt that had built up over the years. As context, the company was founded in 2017.

'During this week, three frontend engineers built out the shell component, and wrapped a fairly isolated use case to be the first Micro Frontend. The team chose the navigation component which sits on top of all pages on the web application, and the code was already isolated enough. The same week, the team also extracted another, smaller feature as the second Micro Frontend. In both cases, there was little refactoring to do, as most of the work was moving the existing code into a new structure.

'Educating the rest of the team on Micro Frontends followed. Instead of jumping to one big refactor, the engineers who did the initial refactoring educated colleagues on why Micro Frontends are useful and how they can be used. The team shared documentation, sent around video tutorials and hosted a few "lunch and learn" sessions. About a month later, all frontend engineers were up to speed on the concepts.

'Going forward, the idea is to build new features for the application in their own Micro Frontend. Doing this from the start is a lot easier, versus going back and refactoring existing code. During the past year, a handful of new Micro Frontends were added, as the product grew.



The structure of the Birdie web app, after introducing Micro Frontends. Over time, each feature can become its own Micro Frontend, and Main navigation and Side navigation are also their own components.

'The process of breaking down the 'legacy' application into Micro Frontends was slower than we expected. The reason for this was two-fold:

1. The concept of 'single responsibility' was something not all parts of the app adhered to. As a startup, we need to ship fast, which made us do trade-offs within the application. When we built the first SPA, we never imagined we would need to break it up into smaller applications. It's because of this that unpicking the features has been trickier, but by no means impossible; it's just more work than copying existing code from one file to another.
2. We were reliant on Redux for managing our global state. When the first SPA was first built, we made use of Redux and Redux-Saga – a Redux side-effect manager – to their full advantage. At the time, this was the right decision, however, it means that lots of our business logic is intertwined, and unpicking a feature to its own component often takes a lot of refactoring and rewriting.

‘Going forward, we are removing cross-feature dependencies on Redux, of which a good example is our analytics code, which used to record user interactions based on Redux actions.

‘Still, we’re keeping in mind what the main goal of our application is. Users should be able to complete the jobs they need to. This is our first measurement of success and everything, including app architecture, comes after this.’

Read the full article where Steve gives more detail on how the developer experience (DX) improved with the move to Micro Frontends:

Autonomy for teams was another reason Birdie moved to Micro Frontends – and I’m not surprised they did so. I see an interesting parallel between microservices for backend teams and Micro Frontends for frontend teams, in how these approaches help teams be more autonomous.

The problems with monolithic backend or frontend applications are that code is tightly coupled, tests can take ages to run, and making a single change in the application might break something, somewhere unexpected.

Both Micro Frontends and microservices introduce more deliberate interfaces between parts of the application. And as long as teams respect these interfaces, they can move faster within their parts of the code and worry less about other Micro Frontends. But an obvious downside can be multiple teams “reinventing the wheel,” or using different approaches to building their respective Micro Frontends. Then again, it’s hard to have both autonomy and shared ways of working.

Steve noted something interesting during our conversation, that monorepos can reduce the cognitive load of context-switching, not just with microservices but also with Micro Frontends. By having all the code of various components in the same codebase – which makes it simple for any engineer to browse them and make changes – it’s possible to naturally end up with similar practices across teams, especially if there’s code reviews taking place across teams working on different Micro Frontends.

As with any architectural choice, there are tradeoffs in each approach. I found Birdie’s use-case interesting, especially as it was the number of tests run during changes which

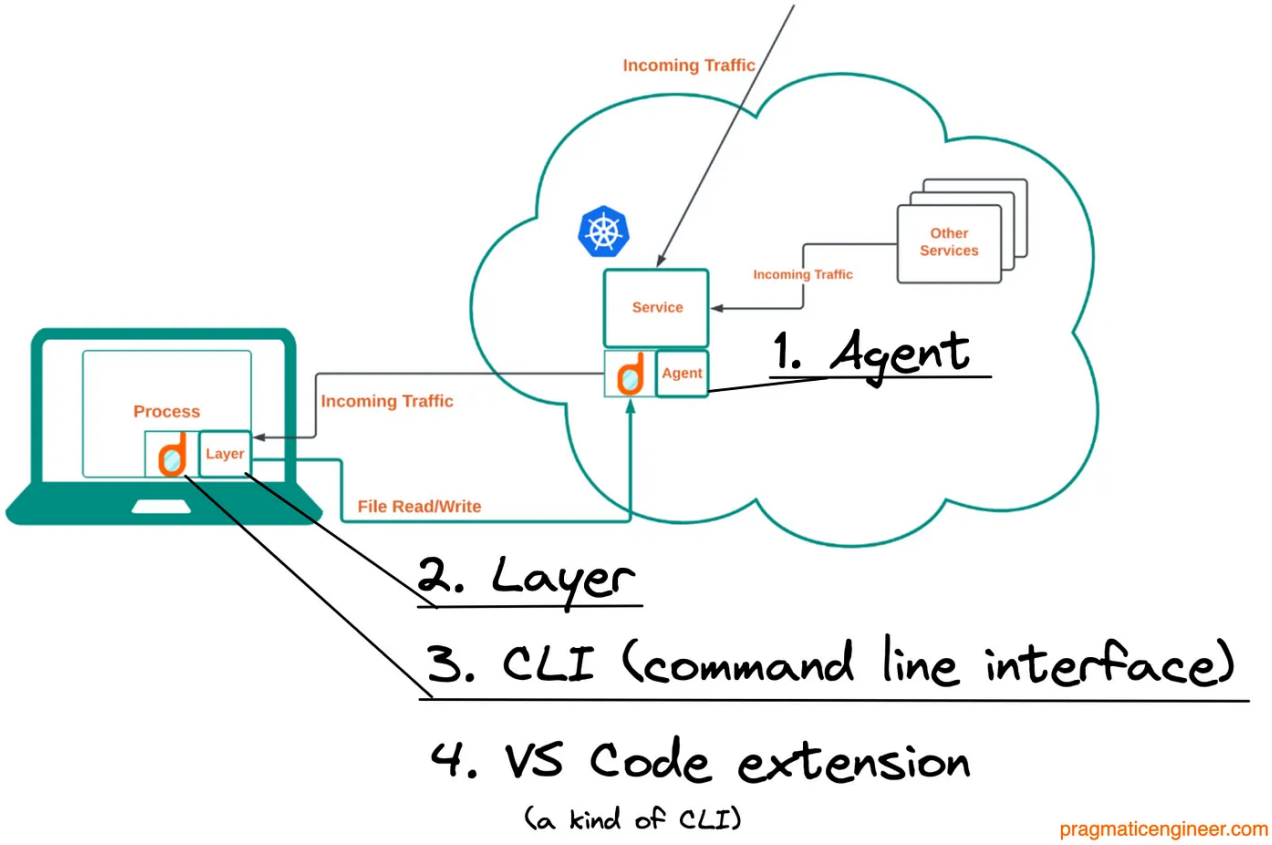
triggered the search for options for splitting up the codebase.

3. Why MetalBear settled on using Rust

Metalbear is a startup building open-source tools for backend developers. The company was only founded in April of this year and is a team of 6 software engineers distributed globally, in Brazil, Canada, Germany and Israel. It's raised around $1M of pre-seed funding.

Their first product is mirrord. The software lets software engineers run local processes, such as their staging environment, in cloud environments without the hassle of deploying to staging. I learned about the company after cofounder, Aviram Hassan, wrote a blog post on why they settled on Rust. I find the experience of a small startup to be an interesting case study.

When starting to develop mirrord, the team chose no one language upfront. Instead, three of its four main components – Agent, Layer, CLI and VS Code extension – each arrived at wanting to use Rust for different reasons.



The four components of mirrord. Source of original image: MetalBear blog.

Here are the considerations for each component:

1. Agent: the component acting as the proxy for users

- **Namespace switching.** Namespace refers to Linux namespaces and networking namespaces. mirrord connects local processes to a remote (Kubernetes) pod, which it does by spawning an agent on the same node as the existing pod. Then, the agent spawns specific threads in the same namespaces of the existing pod. By using the same namespace, the spawned agent can access the same network resources, file resources and process resources. Working with Linux namespaces is easier in some languages than others. It's easier with Rust.
- **Small memory footprint**. Multiple developers have to work in the same environment by minimizing the performance impact. To do so, a small memory footprint is needed, meaning using a language without the overhead of the memory garbage collector. Rust is one such language.

- **Thread safety.** Data needs to be safely moved between threads, so the team needed a language with primitives around safe concurrency and task management. Rust supports Send for thread-safe type transfer and Sync for types to share references across threads.

2. Layer: the shared library running within the local service. Hooks up input/output operations and proxies those to the agent.

- **Low-level requirements.** Due to the need to manipulate sockets and things at the filesystem-level, a low-level language felt like the right choice for the team.
- **Small memory footprint. Similar to the Agent, the goal was to minimize memory usage.**

3. CLI: injecting the layer into the target process.

- **Standalone binaries.** The team looked for a language where the CLI would be generated as a standalone executable, ideally with no dependencies.
- **Future-proof.** Right now, implementing the CLI would have been trivial using pretty much any language. However, looking ahead to more sophisticated injections like using the Unix system call ptrace, a lower-level language like Rust felt more future-proof.

4. VS Code extension: a CLI for Visual Studio Code

- **JavaScript.** VS Code supports only JavaScript, so the choice was easy.

Making hiring easier by being "Rust-only." As a startup, technology can make it significantly easier – or harder! – to hire software engineers. As Aviram puts it:

> "I had a feeling that having our codebase be mainly in Rust would make hiring engineers a lot easier. As a Rust enthusiast, I would love to work somewhere where I'd get to work with Rust regularly, and I suspected that many others felt the same."

Read the full article for more details on how Rust is used in each part of the product – and why.

MetalBear's story is a welcome reminder that early technology choices shape a company's engineering culture – and hiring prospects! The fact is, MetalBear could have chosen to write most of its software in C++, or Go, or C#, Java, or pretty much any other language in which code can be optimized to be performant. I've no doubt they *could* have made mirrord work in any language, even if some would need more workarounds – including possibly going to Assembly-level function hook-ups and then tweaking performance.

However, by selecting a language in a purposeful way, the company made a choice which will have a lasting impact on its engineering culture, hiring, and future choices.

It feels like Rust is gaining popularity thanks to striking a good balance between offering low-level features in a more friendly way than, for example, C or C++ do. It's also a language many backend engineers want to learn and taking this into consideration is a smart move; thinking beyond the hiring of the first software engineers.

4. Why Motive moved over to Kotlin Multiplatform Mobile

Motive – formerly KeepTruckin – is an automated operations platform for the logistics sector. They build a variety of software products that aid trucking operations, fleet management and other use cases where hardware and software can help improve the way shipping businesses work. The company raised a $150M Series F funding in May this year.

Sunil Kumar, staff software engineer at the company, wrote a blog post this July about why the engineering team decided to move to Kotlin Multiplatform Mobile (KMM) as an approach for sharing more code between Android and iOS. He also detailed their experience with the change.

The engineering team building the Motive Fleet application wanted to ensure consistency in business logic across the mobile apps, and to execute faster on development They evaluated three cross-platform development approaches:

- Flutter: a cross-platform framework built by Google using the Dart programming language, which Google also developed.
- React Native: a cross-platform framework originally built by Facebook. It uses the JavaScript programming language and has some similarities with the React web framework.
- KMM: A framework built by Jetbrains. It uses the Kotlin programming language.

The team did further comparisons, looking at the dimensions of UI support and how easy it is to build native UI, to integrate with existing applications, and performance implications. Here's what the team evaluated:

Criterion	KMM	Flutter	React Native
Language	Kotlin	Dart	Javascript
Performance	Native performance	Native performance	Performance not as good as Native
Integration	Simple library integration	Requires deep integration	Requires deep integration
UI	Native UI	Flutter UI Kit	UI components written using Javascript

Comparing KMM, Flutter and React Native on dimensions that are important for the Motive engineering team. Source: Motive engineering blog.

The team decided to go with KMM, mostly because it is the easiest to work with their existing code and they felt they could keep the most (native) control of the app.

I reached out to Sunil with questions on how they came to the decision. Here's what he shared:

Which other reasons led to KMM?

'The Motive Fleet app team was built just a few days before Covid-19 started. However, we were short on iOS bandwidth, which led to iOS being always lagging 1-2 months behind Android. This lag meant some features had discrepancies in the business logic between apps.

‘To get rid of these differences and to speed up the development due to resource shortage, we started looking into cross-platform solutions. I had previous experience with React Native. When I worked with React Native, the app’s performance that I built was not great. In our case, performance was a key point as we interact with maps a lot. With Flutter, no one in our team had any experience with it and after reading multiple blog posts, we decided it wasn't worth the effort of learning a new technology and a new language.’

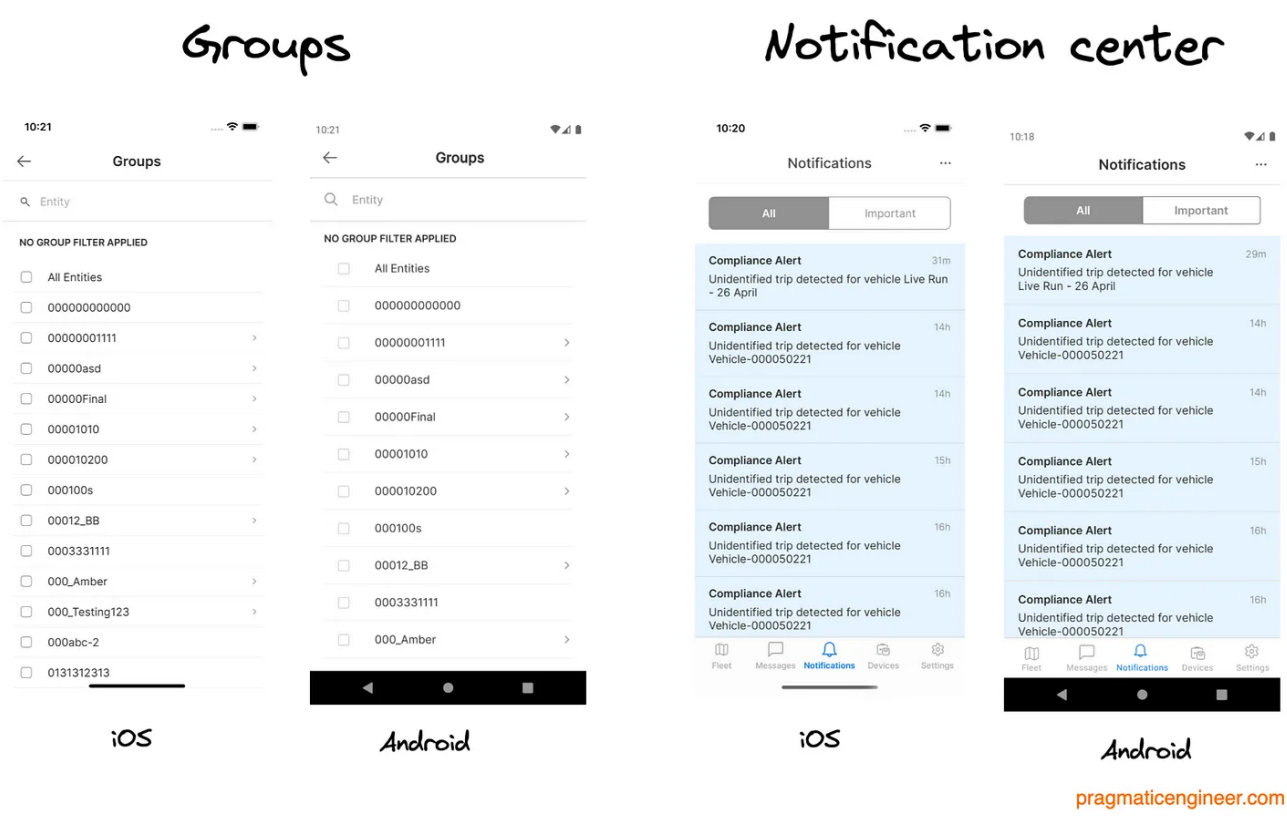
How did you coordinate the move over to KMM?

‘We did not migrate all of the existing app’s code to KMM. Instead, we started small. We wrote new code for the network layer using KMM and moved over the session management to KMM by removing the existing JVM (Java Virtual Machine) dependencies. Once this work was done, we only used KMM for new features we built.

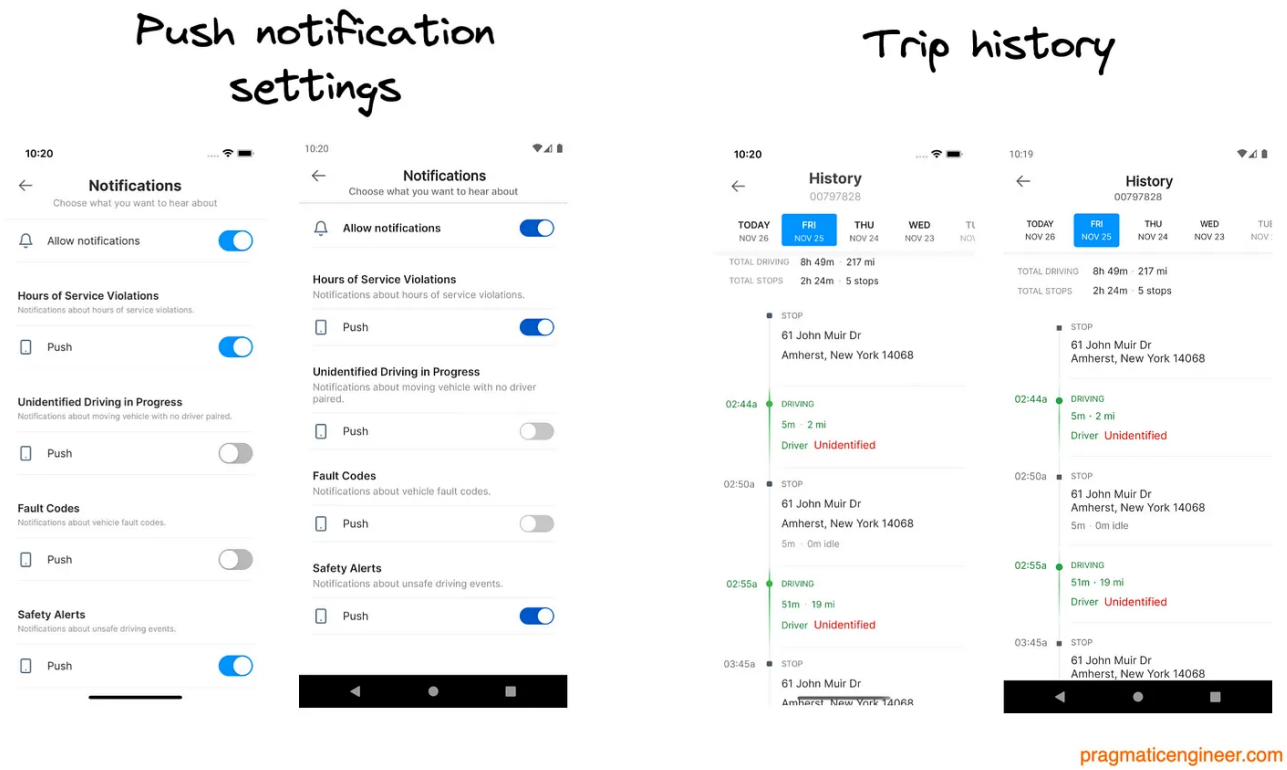
‘So far, the features in the app that use KMM are:

1. **Notification center.** This is where users can get an overview of all the alerts about their fleets. This component uses the shared SQL database.
2. **Trip History.** Users can see the trip history of vehicles, drivers and assets.
3. **Push Notification Settings**. Users can control which alerts they want to receive Push notifications for, on their devices.
4. **Groups**. Users can filter their fleet data, based on different user-created groups.’

Here’s a screenshot of what these features look like, across iOS and Android:



Groups and Notification center: both features written using Kotlin Multiplatform Mobile.



Push notification settings and Trip history in the Motive Fleets app. The UIs are nearly identical on iOS and Android.

Read the full article for details on the KMM architecture the team chose, the code structure they put in place, and code-level details of how they integrated the KMM module in the existing Motive Fleet apps on iOS and Android:

Building separate iOS and Android apps leads to the question: 'can we not just use shared code?' This is something many CEOs, product folks and even many mobile engineers will ask. As for customers, they generally don't care which technologies a mobile app uses, and expect roughly the same functionality across iOS, Android and often the web.

A month ago, we explored whether there's a drop in native iOS and Android hiring at startups. The answer is nuanced, but I did note that:

> 'Today, Flutter and React Native are finally "good enough" for apps which don't have high performance demands.'

In the case of Motive, performance was important enough to not jump on to React Native or Flutter, nor were they ready to throw away existing code they'd written and jump onto a different technology stack.

Still, introducing KMM is no small matter, as now a good portion of native development is done with Kotlin, which is a language iOS and Android engineers both have to familiarize themselves with. And in the case of mobile apps, the benefit of not having to test two separate business logic implementations thoroughly on OS and Android, is that it saves time on testing and reduces potential for business logic inconsistencies to creep in.

I asked Sunil how he feels about the move and putting in all the work to refactor the code, in order to take advantage of the KMM stack. He said he's happy with the decision and estimated their development cycle is now about a third faster than it was, thanks to the unified Android/iOS business logic. Most crucially, they no longer have to wait on iOS, even when they have less iOS bandwidth.

Takeaways

This Real-World Engineering issue focuses on case studies where teams chose new technologies to solve existing pain points. With a variety of examples, I hope it highlights the process of choosing a new technology and how teams handle the move.

We've covered case studies about choosing a new messaging queue, refactoring a web app to a new architectural model, choosing a language to be used across a startup, and moving over to shared iOS and Android business logic. Factors that stand out to me as approaches worth considering, include:

- **When changing technologies, make sure you solve for a large enough pain point.** Changing technologies – such as replacing a framework or choosing a new architecture approach – are expensive changes in terms of time and complexity. Make sure your pain point is significant enough to warrant this change. In the case of Trello, the combined issue of reliability of web sockets and overly high resource costs, made it worth it. Does this hold true in your use case, too?
- **When building a new project, think about future pain points.** When you're starting to build something new, you need far less justification for choosing any technology as you don't have any cost to pay for switching. However, try to choose a technology which helps you avoid – or solve for – future pain points. This is what MetalBear has done by anticipating that Rust will make a lot of their future use cases easier to tackle, on top of being a good fit right now.
- **When changing technologies, do it one step at a time, if possible.** When moving over to Micro Frontends and when introducing KMM, the team first prototyped the approach, then shipped one feature using it, and then started to build new features with the new approach.
- **Going back to refactor everything is not necessarily a pragmatic approach.** When changing technologies completely – such as choosing a new messaging approach – you might have no choice but to replace your existing technology. However, when changing architectural approaches, or making large alterations, an "everything must go" approach might make more sense. Both Birdie with Micro Frontends and Motive with KMM, have not moved all their existing code over to the new approach or framework, at least not yet.

However, my biggest takeaway is this:

Don't forget your #1 priority, as an engineering team. What is the biggest measure of success for your team? For Birdie, Steve Heyes said it's that "users should be able to complete the jobs they need to." Answer the question for yourself and make sure you keep that priority in mind: it surely comes before the technology you choose to work with.

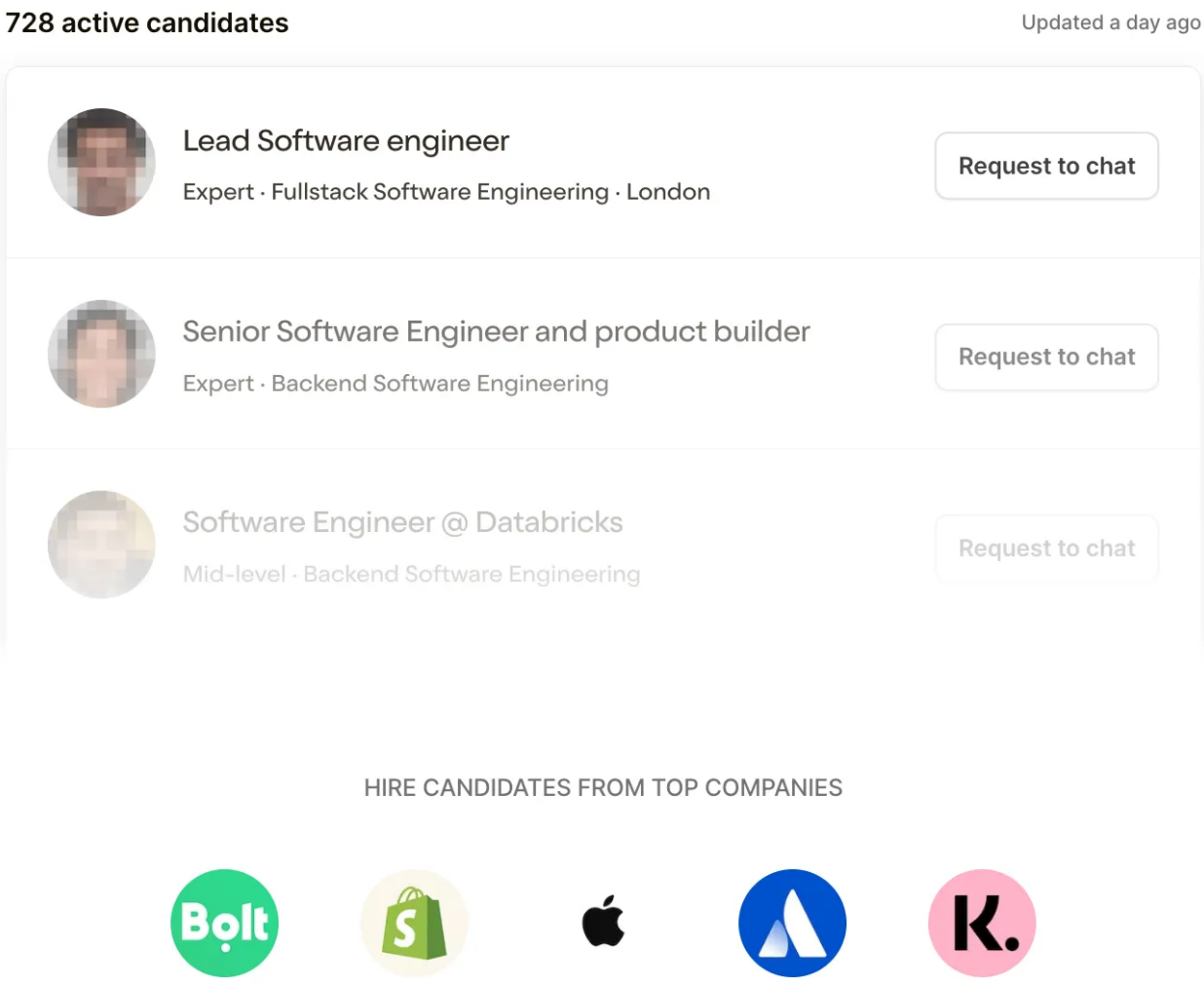
Featured Pragmatic Engineer Jobs

1. Senior Full Stack/Frontend Engineer at Vitally.io. **$180-270K**. New York or Remote.
2. Engineering Manager at Gruntwork. **$175-240K** + equity. Remote (Global).
3. Founding Engineer at Renterra. **$140-180K** + equity. Remote (Global).
4. Machine Learning Engineering Lead at Conjecture. **£85-210K** + equity. London (UK).
5. Full Stack Software Engineer at Insitro. Poland.
6. Staff Back-End Engineer - Core Services at BetterUp. Remote (Germany, Netherlands or the UK).
7. Senior Lead Software Engineer - Kubernetes at Akamai Technologies. Remote (US).
8. Senior Software Engineer - Cloud Native at Akamai Technologies. Remote (US).
9. Software Engineer at DevZero. **$150-175K**. Seattle, Washington.
10. Senior Backend Developer at Founda Health. Amsterdam, Netherlands.
11. Senior Backend Engineer at Vital. **$70-140K** + equity. Remote.
12. Principal Backend Enginee at Pento. **£120-135K**. Remote.
13. Founding Senior Fullstack Engineer (JavaScript) at Playht. **$150-200K** + equity. San Francisco or Remote.
14. Staff Software Engineer at Qualified.com. San Francisco or Remote.
15. Infrastructure Team Lead at Ometria. **£90-150K**. United Kingdom or Portugal.

See more senior engineer and leadership roles with great engineering cultures on [The Pragmatic Engineer Job board](#) - or post your own.

Join The Pragmatic Engineer Talent Collective

If you're hiring, join [The Pragmatic Engineer Talent Collective](#) to start getting regular drops of outstanding software engineers and engineering leaders who are open to new opportunities. It's a great place to hire developers - from backend, through fullstack to mobile - and engineering managers and executives.



If you're open for a new opportunity, join to get reachouts from vetted companies. You can join anonymously and leave anytime

120 Likes

3 Comments

Write a comment...

Martin Gallauner Nov 30, 2022

Good read. Never heard about KMM before tbh, I need to read into that!

LIKE (1) REPLY ...

Ben Butterworth Feb 13

> With Flutter, no one in our team had any experience with it and after reading multiple blog posts, we decided it wasn't worth the effort of learning a new technology and a new language.'

Reading multiple blog posts should not be how you decide that a technology is *not worth it*. Especially one that is incredibly hyped. There are plenty of people who do think that it is worth learning it from scratch. Just observe the stats that Flutter regularly share, e.g. in Flutter Forward.

I am probably a minority of people who have heard of KMM (Kotlin Muliplatform Mobile), but as someone who works on mobile apps and is trying to be observant about the industry, I don't see it being used or talked about. I think that is a huge red flag. It's been around for quite a few years, but only reached beta 4 months ago. In mobile development, there are plenty of issues which your programming language doesn't solve: deployment to app stores, test automation, screenshot automation for store listings, offline, push notifications - in fact Gergely wrote a book on it - "Building Mobile Apps at Scale". Look at the announcement of KMM Beta, 7 points on HN: https://news.ycombinator.com/item?id=33152772. Look at the announcement of Flutter 3: 604 points on HN: https://news.ycombinator.com/item?id=31344863. Don't use points for everything, but

100x is a big deal - I'd guess there was 1000x more Flutter apps on the stores than KMM. 😂

Jetbrains (creator of Kotlin and KMM) have recently announced they will kill of AppCode, their Swift/iOS/macOS IDE. That is to say, they are not committed to iOS or macOS - I think this is a silly move but unfortunately it wasn't profitable enough for them. Perhaps they're diverting resources to KMM. Google have been heavily investing in Flutter. As such, the common "Google-will-kill-it" argument actually applies to Jetbrains more than Google.

I would say Kotlin is a nicer programming language than Dart. However, my gut feeling is that the companies with KMM Apps that survive will probably be re-written in Flutter or just native Android/iOS in the next few years.

Disclosure: I have written Kotlin professionally but never used KMM. I have also worked on Flutter apps and packages. Flutter + Kotlin would be nice 😅 - I don't think this will happen though.

LIKE REPLY ...

1 more comment...